MWS GPT (ex. LLMOps Platform)

MWS GPT (ex. LLMOps Platform)

Exported on 04/02/2026

Table of Contents

1 Как начать работу?....10
2 FAQ....11
3 Полезный контент:....12
3.1 Полезные ссылки:....12
3.1.1 Внутренние:....12
3.1.2 Внешние:....12
3.1.3 API Документация:....12
3.1.4 Внутренняя:....12
3.1.5 Внешняя:....13
3.2 Пользовательская документация:....13
3.3 Записи вебинаров:....13
4 Все у вас получится....15
5 Коммерция....16
5.1 Коммерческие материалы....16
5.1.1 GTM....16
5.1.2 Системные требования....16
5.1.3 Sales Deck....16
5.1.4 Список конкурентов....17
5.1.5 Внешние исследования рынка....17
5.1.6 Заказ (MWS)....17
5.1.7 Sales Kit (MWS)....17
5.1.8 Спец. условия облачного сервиса (MWS)....17
5.1.9 Расчет маржинальности (MWS)....18
5.1.10 Тарифы (MWS)....18
5.1.11 Чек-лист по передаче продукта на ТП (MWS)....18
5.1.12 Onepager (MWS)....19
5.2 Внешняя документация....19

6 Разработка 20
6.1 Product Backlog Refinement 20
6.2 QA 22
6.2.1 Core 22
6.2.2 No-Code 22
6.2.2.1 Агенты 22
6.2.2.2 Компоненты No-code 22
6.2.2.3 Модели 25
6.2.2.4 Рабочие сценарии 25
6.3 VLM 38
6.3.1 Бенчмаркинг 38
6.3.1.1 LLaVaRU 41
6.3.1.2 LLaVaTranslated 42
6.3.1.3 Text-only benchmark: 43
6.3.2 Исследования 43
6.3.3 Таблица экспериментов 44
6.4 Анализ задержки запуска Flow в Langflow 65
6.4.1 Общая схема выполнения 65
6.4.2 Выявленные проблемы 66
6.4.2.1 1. Граф пересоздаётся с нуля при каждом запуске 66
6.4.2.2 2. Deep copy всех данных flow 67
6.4.2.3 3. Последовательное инстанциирование компонентов 67
6.4.2.4 4. Двухфазный HTTP-запрос на фронтенде 68
6.4.2.5 5. Загрузка flow из БД (большой JSON-блоб) 69
6.4.2.6 6. Инициализация сервиса трассировки 70
6.5 Как работают связи компонентов в воркфлоу Langflow и почему они ломаются при обновлении версии 70
6.5.1 1. Как устроены связи компонентов 70
6.5.1.1 1.1 Модель данных 70

6.5.1.2 1.2 Что хранится в ноде 71
6.5.1.3 1.3 Хранение в базе данных....71
6.5.1.4 1.4 Создание связи на фронтенде 72
6.5.1.5 1.5 Последовательность загрузки flow 72
6.5.1.6 1.6 Валидация на фронтенде - cleanEdges()....73
6.5.1.7 1.7 Валидация на бэкенде 73
6.5.1.8 1.8 Система шаблонов и определение устаревших компонентов 74
6.5.2 2. Где ломается при обновлении 74
6.5.2.1 2.1 Сценарий обновления....74
6.5.2.2 2.2 Точка отказа 1 - строковое сравнение в cleanEdges() 75
6.5.2.3 2.3 Точка отказа 2 - output не найден по имени....75
6.5.2.4 2.4 Точка отказа 3 - несовпадение output_types в сериализованном handle 75
6.5.2.5 2.5 Точка отказа 4 - скрытые поля удаляют edge-и 76
6.5.2.6 2.6 Точка отказа 5 - валидация на бэкенде при запуске 76
6.5.2.7 2.7 Почему это тихо 76
6.5.2.8 2.8 Почему это необратимо....76
6.5.3 3. Найденные баги 77
6.5.3.1 Баг 1: cleanEdges() сравнивает строки вместо проверки совместимости типов 77
6.5.3.2 Баг 2: шаблоны не обновляются до очистки edge-ей 77
6.5.3.3 Баг 3: auto-save делает потерю edge-ей необратимой 77
6.5.3.4 Баг 4: detectBrokenEdgesEdges() и cleanEdges() по-разному ищут source output....78
6.5.3.5 Баг 5: selected_output ссылается на удаленный output 78
6.6 Компоненты LLMOps....79
6.7 Описание требований и архитектуры 79
6.7.1 Копия Stack [AS IS]....79
6.7.1.1 Текущий стек реализуется компонентами:....79
6.7.2 Сценарии использования 82

6.7.2.1 Сценарии использования LibreChat [UI] 82
6.7.2.2 Сценарии использования LiteLLM [UI] 107
6.7.2.3 Сценарии использования NoCode aka LangFlow [UI] 107
6.8 Передача репозитория в деплой (Infra / DevOps) 107
6.8.1 Цель документа 107
6.8.2 Минимальные требования к сервису 107
6.8.2.1 1. Healthcheck endpoint 107
6.8.2.2 2. Метрики (если применимо) 108
6.8.2.3 3. Алерты (при интеграции с инфровым алертингом) 108
6.8.3 Требования к репозиторию 108
6.8.3.1 4. Репозиторий с кодом 108
6.8.3.2 5. Ссылка на репозиторий в задаче 108
6.8.4 Обязательная структура репозитория 109
6.8.4.1 6. Dockerfile 109
6.8.4.2 7. Файл зависимостей 109
6.8.4.3 8. README.md (обязательно) 109
6.8.4.4 9. Production docker-compose (если требуется) 110
6.8.5 Процесс передачи в Infra / DevOps 110
6.9 Ролевая модель в Langflow (NoCode) 111
6.9.1 Цели создания ролевой модели 111
6.9.2 Открытые вопросы 111
6.9.3 Основные сущности 111
6.9.4 Роли 112
6.9.4.1 Админ организации 112
6.9.4.2 Админ проекта 112
6.9.4.3 Тестировщик (Tester / Reviewer) 113
6.9.4.4 Пользователь проекта 113
6.9.4.5 Наблюдатель проекта 113
6.9.5 Схематичное изображение ролевой модели 115

6.9.6 RBAC в Langflow с использованием Casbin ... 115
6.9.6.1 1. Что сейчас есть ... 115
6.9.6.2 2. Выбранный подход ... 116
6.9.6.3 3. Два варианта реализации ... 117
6.9.6.4 4. Вариант А (MVP) - что конкретно делаем ... 118
6.9.6.5 5. Вариант Б (полный) - что добавляется сверх Варианта А ... 123
6.9.6.6 6. Матрица ролей и действий ... 124
6.9.6.7 7. Переменные окружения ... 126
6.9.6.8 8. Какие файлы Langflow меняются ... 127
6.9.6.9 9. Как проходит запрос (Вариант А) ... 127
6.9.6.10 10. Риски и митигация ... 128
6.9.6.11 11. Плюсы и минусы выбранного подхода ... 129
6.9.6.12 12. Миграция с Варианта А на Вариант Б ... 130
6.9.7 Анализ кодовой базы по ролевке LF и оценка предлагаемых решений ... 130
6.9.7.1 1. Текущее состояние архитектуры Langflow ... 130
6.9.7.2 2. Требования к решению для форка ... 132
6.9.7.3 3. Четыре подхода к интеграции RBAC ... 132
6.9.7.4 Выводы ... 136
6.9.7.5 5. Почему Casbin подходит для RBAC в Langflow ... 136
6.9.7.6 6. Интеграция требований ролевой модели с Casbin ... 137
6.9.7.7 7. Заключение ... 138
6.9.8 Проектирование UI-ролевки для NoCode ... 139
6.9.8.1 Ссылка на макет ... 139
6.9.8.2 Введение ... 139
6.9.8.3 Основные принципы проектирования ... 140
6.9.8.4 Описание пользовательских ролей ... 142
6.9.8.5 Сценарии использования ... 145
6.9.8.6 Раздел UI-архитектура: страницы и компоненты ... 148

6.10 Чат MWS GPT 155
6.10.1 Интеграция получения статуса премиум-подписки через MTS ID 155
6.10.1.1 Ссылки 155
6.10.1.2 Описание задачи 156
6.10.1.3 Реализация 156
7 Техническая поддержка MWS GPT 167
7.1 FAQ 167
7.1.1 Как начать работу? 167
7.1.2 Какие модели у вас есть? 167
7.1.3 Какая модель лучше для ... ? 167
7.1.4 Отправляю запрос с локального компьютера из внутренней сети, а он не проходит 168
7.1.5 Отправляю запрос а он ругается на сертификаты 168
7.1.6 Как сделать RAG ? 168
7.1.7 Как сделать агента ? 169
7.1.8 Почему запрос обрабатывается так долго? 169
7.1.9 Модель лежит, я получаю ошибку APITimeoutError: Request timed out, что делать? 169
7.1.10 Как посмотреть доступность платформы? 170
7.1.11 Кто может проконсультировать по внедрению LLM в мой продукт? 171
7.1.12 Где посмотреть полезный контент по внедрению LLM? 171
7.1.13 Как понять, какая LLM локальная, а какая внешняя? 171
7.1.14 Как понять, какие LLM можно использовать с чувствительной информацией? 171
7.1.15 У меня техническая проблема/нужна консультация 172
7.1.16 У меня ключ старого формата (ASGK), что делать? 172
7.1.17 У меня еще остались вопросы, где задать? 172
7.2 Как начать работу? 173
7.2.1 Получение ключа 173
7.2.1.1 Заполнение формы в MWS Tables 174

7.2.1.2 Как заполнять поля в форме? 174
7.2.2 Настройка доступов на Firewall 175
7.2.2.1 Заявка на Firewall 176
7.2.3 Отправляем запросы в MWS GPT 177
7.2.3.1 Спецификация OpenAI API 177
7.2.3.2 Postman коллекция 177
7.2.3.3 Библиотеки для интеграции 177
7.2.3.4 Интеграция с 3rd-party сервисами и библиотеками 178
7.2.4 Я все сделал, но у меня еще остались вопросы 178
7.3 Модели и Бенчмарки 179
7.4 Чек-лист технологической зрелости MWS GPT 183

MWS GPT — это облачная B2B-платформа от МТС Web Services (MWS), предназначенная для работы бизнеса с большими языковыми моделями (LLM). Она позволяет использовать, дообучать и внедрять как собственные, так и сторонние (open source) модели, предлагая инструменты вроде чат-интерфейса и no-code конструктора для создания приложений.

Основные функции и возможности:

- **Доступ к LLM:** Платформа объединяет и предоставляет доступ к различным большим языковым моделям, включая собственные разработки МТС и модели с открытым исходным кодом.
- **Работа с моделями:** Пользователи могут дообучать модели на своих данных и внедрять их в бизнес-процессы через инфраструктуру MWS.
- **Удобный интерфейс:** Включает чат, который позволяет сохранять и обмениваться запросами (промптами), а также no-code конструктор для создания приложений без привлечения программистов.
- **Агентская архитектура:** В основе платформы лежит агентская архитектура, которая позволяет моделям самостоятельно выбирать необходимые навыки для решения конкретной задачи.
- **Гибкая оплата:** Стоимость зависит от количества использованных токенов, что делает платформу доступной для компаний разного размера, включая малый и средний бизнес.
- **Инфраструктура:** Работает на облачной инфраструктуре МТС, что обеспечивает отказоустойчивость и масштабируемость, и планируется возможность развертывания на серверах заказчика.

Наш сайт: https://mws.ru/services/mws-gpt/

1 Как начать работу?

Как начать работу? (see page 173)

2 FAQ

FAQ (see page 167)

3 Полезный контент:

3.1 Полезные ссылки:

3.1.1 Внутренние:

Внутренний UI Chat - https://ui.llmops.mts-corp.ru/login

Внутренний No-Code конструктор - https://langflow.llmops.mts-corp.ru/login

Внутренний API - https://api.llmops.mts-corp.ru/

Внутренний мониторинг - https://grafana-gbp.genai.mts-corp.ru/login (login: reader; password: reader)

Внутренний продукт - https://product.mts.ru/product/BI_3060/main

3.1.2 Внешние:

Внешний Demo UI Chat - https://ui.gpt.mws.ru/login

Внешний Demo No-Code конструктор - https://nocode.gpt.mws.ru/login

Внешний B2C Chat - https://gpt.mws.ru/

Внешний API - https://api.gpt.mws.ru/

Внешний продукт - https://product.mts.ru/product/BI_3508/main

3.1.3 API Документация:

3.1.4 Внутренняя:

API Документация-v4-20250210_115130.pdf[1]

1 https://confluence.mts.ru/download/attachments/1227141129/API%20%D0%94%D0%BE%D0%BA%D1%83%D0%BC%D0%B5%D0%BD%D1%82%D0%B0%D1%86%D0%B8%D1%8F-v4-20250210_115130.pdf?api=v2&modificationDate=1763109694231&version=1

3.1.5 Внешняя:

API Документация mws-v1-20250210_150202.pdf[2]

3.2 Пользовательская документация:

3_1_Инструкция_по_установке_и_информацией,_необходимой_для_эксплуатации.pdf[3]

3.3 Записи вебинаров:

Вебинар_модели ИИ_зоопарк_Аня Щеникова[4]

Курс по искусственному интеллекту[5]

Мастер-класс по работе с искусственным интеллектом[6]

Митап True Tech AI - июль 2024[7]

Митап по GenAI от Центра RnD TechGov[8]

Prompt Engineering — как найти общий язык с LLM | Наталья Бруй и Алёна Шувар | True Tech Day 2.0[9]

ML в эпоху LLM. Доклад Даниила Киреева | True Tech Champ 2024[10]

Как мы адаптировали токенайзеры вместе с ВШЭ — и нашли баланс между скоростью генерации и качеством русской речи | TTD 2025[11]

10 правил, как проектировать LLM-приложение. Доклад Всеволода Викулина | True Tech Day 2025[12]

Ландшафт LLM. модели, бенчмарки, применение. Доклад Даниила Киреева | True Tech Day 2025[13]

Реальные прикладные кейсы использования LLM. Доклад Павла Бабина | True Tech Day 2025[14]

---

2 https://confluence.mts.ru/download/attachments/1227141129/API%20%D0%94%D0%BE%D0%BA%D1%83%D0%BC%D0%B5%D0%BD%D1%82%D0%B0%D1%86%D0%B8%D1%8F%20mws-v1-20250210_150202.pdf?api=v2&modificationDate=1763109660080&version=1

3 https://confluence.mts.ru/download/attachments/1227141129/3_1_%D0%98%D0%BD%D1%81%D1%82%D1%80%D1%83%D0%BA%D1%86%D0%B8%D1%8F_%D0%BF%D0%BE_%D1%83%D1%81%D1%82%D0%B0%D0%BD%D0%BE%D0%B2%D0%BA%D0%B5_%D0%B8_%D0%B8%D0%BD%D1%84%D0%BE%D1%80%D0%BC%D0%B0%D1%86%D0%B8%D0%B5%D0%B8%CC%86%2C_%D0%BD%D0%B5%D0%BE%D0%B1%D1%85%D0%BE%D0%B4%D0%B8%D0%BC%D0%BE%D0%B8%CC%86_%D0%B4%D0%BB%D1%8F_%D1%8D%D0%BA%D1%81%D0%BF%D0%BB%D1%83%D0%B0%D1%82%D0%B0%D1%86%D0%B8%D0%B8.pdf?api=v2&modificationDate=1763118539344&version=1

4 https://confluence.mts.ru/pages/viewpage.action?pageId=1102153391

5 https://confluence.mts.ru/pages/viewpage.action?pageId=1252747135

6 https://confluence.mts.ru/pages/viewpage.action?pageId=1244788534

7 https://confluence.mts.ru/pages/viewpage.action?pageId=1230080153

8 https://confluence.mts.ru/pages/viewpage.action?pageId=1071925503

9 https://www.youtube.com/watch?v=-epG5fF074o

10 https://www.youtube.com/watch?v=Kcp8Yk9K6uI

11 https://vkvideo.ru/video-226874221_456239458?t=7s&ref_domain=truetechday.ru

12 https://vkvideo.ru/video-226874221_456239453?t=1s&ref_domain=truetechday.ru

13 https://vkvideo.ru/video-226874221_456239459?t=1s&ref_domain=truetechday.ru

14 https://vkvideo.ru/video-226874221_456239462?t=0s&ref_domain=truetechday.ru

Как взять LLM за RAGa. Доклад Александра Абрамова | True Tech Day 2025[15]

KODIFY 2.0 — сервис генерации и дополнения кода. Доклад Валентина Малых | True Tech Day 2025[16]

Cost of Ownership: LLM in-house vs SaaS[17]

AI — эволюция облака, а не опция[18]

Онлайн-дискуссия «Как создать AI-агента без кода»[19]

Расскажите про MWS. AI-автоматизация: как построить команду с нуля и перестать перекладывать задачи руками?[20]

MWS GPT: заблуждения и мифы про внедрение LLM в бизнес[21]

Онлайн-дискуссия «Почему не стоит инвестировать 2 года в платформу инференса LLM?»[22]

15 https://vkvideo.ru/video-226874221_456239457?t=1s&ref_domain=truetechday.ru
16 https://vkvideo.ru/video-226874221_456239455?t=0s&ref_domain=truetechday.ru
17 https://vkvideo.ru/video-38818370_456239332
18 https://vkvideo.ru/video-38818370_456239327
19 https://vkvideo.ru/video-38818370_456239300
20 https://vkvideo.ru/video-38818370_456239290
21 https://vkvideo.ru/video-38818370_456239264
22 https://vkvideo.ru/video-38818370_456239241

4 Все у вас получится

5 Коммерция

5.1 Коммерческие материалы

5.1.1 GTM

Версия	Документ
v1.0	

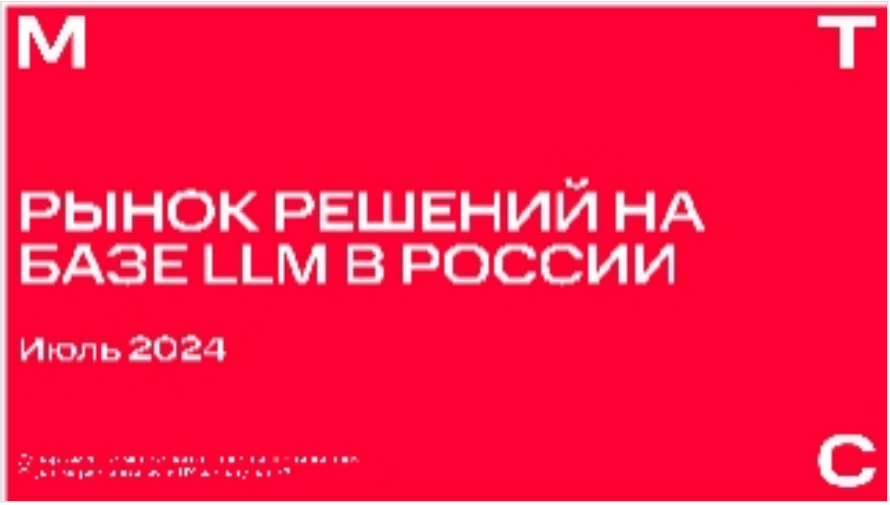
5.1.2 Системные требования

5.1.3 Sales Deck

Версия	Документ

5.1.4 Список конкурентов

5.1.5 Внешние исследования рынка



5.1.6 Заказ (MWS)

Версия	Документ

5.1.7 Sales Kit (MWS)

Версия	Документ

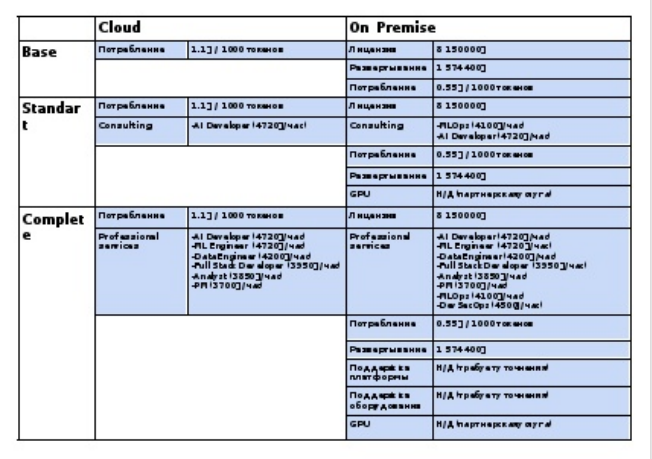
5.1.8 Спец. условия облачного сервиса (MWS)

Версия	Документ

5.1.9 Расчет маржинальности (MWS)

Версия	Документ

5.1.10 Тарифы (MWS)

Версия	Документ
0.1	

	Cloud		On Premise	
Base	Потребление	1.1₽ / 1000 токенов	Лицензия	8 150000₽
			Развертывание	1 574 400₽
			Потребление	0.55₽ / 1000 токенов
Standart	Потребление	1.1₽ / 1000 токенов	Лицензия	8 150000₽
	Consulting	-AI Developer (4720₽/час)	Consulting	-MLOps (4100₽/час) -AI Developer (4720₽/час)
			Потребление	0.55₽ / 1000 токенов
			Развертывание	1 574 400₽
			GPU	N/A (партнерская услуга)
Complete	Потребление	1.1₽ / 1000 токенов	Лицензия	8 150000₽
	Professional services	-AI Developer (4720₽/час) -ML Engineer (4720₽/час) -DataEngineer (4200₽/час) -Full Stack Developer (3550₽/час) -Analyst (3850₽/час) -PM (3700₽/час)	Professional services	-AI Developer (4720₽/час) -ML Engineer (4720₽/час) -DataEngineer (4200₽/час) -Full Stack Developer (3550₽/час) -Analyst (3850₽/час) -PM (3700₽/час) -MLOps (4100₽/час) -DevSecOps (4500₽/час)
			Потребление	0.55₽ / 1000 токенов
			Развертывание	1 574 400₽
			Поддержка платформы	N/A (требуется уточнение)
			Поддержка оборудования	N/A (требуется уточнение)
			GPU	N/A (партнерская услуга)

5.1.11 Чек-лист по передаче продукта на ТП (MWS)

Версия	Документ

5.1.12 Onepager (MWS)

Версия	Документ
v1.0	

5.2 Внешняя документация

6 Разработка

6.1 Product Backlog Refinement

Встречи проводятся еженедельно по ПН.

Цели встречи:

1. Выработать единое понимание приоритетов;
2. Провалидировать запросы со стороны Стейкхолдеров;
3. Определить, какие задачи готовы войти в ближайшие спринты, какие deprecated

Итог встречи:

1. Список задач и статусов к изменению по ним
2. Список открытых вопросов
3. Назначены ответственные по пунктам выше

№ задачи	Статус	Дата запроса	Срок	Отв.
нет	Стабилизация LangFlow			**Горинов Д.**
LLMPLA-312[23]	**librechat & langflow integration**			-
LLMPLA-238[24]	**Биллинг для SaaS**			Зависимость с MWS Console
нет	RateLimit			**Горинов Д.**
нет	BudgetLimit		**не срочно**	Зависимость с MWS Console

23 https://jira.mts.ru/browse/LLMPLA-312
24 https://jira.mts.ru/browse/LLMPLA-238

LLMPLA-239[25]	**Прокси для внешним моделей для image generation и sst/tts** **Пробросить внешние модели**			**Горинов Д.**
LLMPLA-373[26]	Выполнено			Криванков Д.
Нет	**Выдача клиентам реселлерам токенов для их потребителей**			**Криванков Д.**
Нет	**RoadMap доп конца года**			**Горинов Д.**
нет	RateLimit BudgetLimit Завести задачу на исследование	08.09.2025		Игорь Кочелков
нет	Ввести эпик Интеграция с MWS			**Горинов Д.**
нет	Апдейт Виспера			**Горинов Д.**
нет	RnD по биллингу модальностей			**Горинов Д.**
нет	Завести задачу на обновление версии во внешнем стенде LibreChat Версия либо из Инсайда, либо самая последняя у Максима Красильникова Доп: повторить тестирование.	22.09.2025		**Горинов Д.**

25 https://jira.mts.ru/browse/LLMPLA-239
26 https://jira.mts.ru/browse/LLMPLA-373

LLMPLA-596[27]	Нужна отдельная встреча с проработкой данного вопроса. Пока не приоритизируем	15.09.2025		**Криванков Д.**
нет	Добавить Идею: Политика чистки/хранения загруженных файлов в LibreChat	15.09.2025		**Гуров О.**
	Привести в порядок Ocean (Inside)			

6.2 QA

6.2.1 Core

6.2.2 No-Code

6.2.2.1 Агенты

6.2.2.2 Компоненты No-code

- Agents (see page 22)
- Data (see page 23)
- Helpers (see page 23)
- Input/Output (see page 23)
- Logic (see page 23)
- Processing (see page 23)
- Vector stores (see page 24)

6.2.2.2.1 Agents

Agent

MCP Tool

27 https://jira.mts.ru/browse/LLMPLA-596

6.2.2.2.2 Data

Api Request

Directory

File

News Search

RSS Reader

SQL Database

URL

Web Search

Webhook

6.2.2.2.3 Helpers

Calculator

Current Date

Message History

6.2.2.2.4 Input/Output

Chat Input

Chat Output

Text Input

Text Output

6.2.2.2.5 Logic

If-else

Listen (beta)

Loop

Notify (beta)

Run Flow (beta)

6.2.2.2.6 Processing

Prompt Template

Batch Run

Data Operations

Dataframe Operations

LLM Router

Parser
Python Interpreter

Save File

Smart Function

Split Text

Structured Output

Type Convert

6.2.2.2.7 Vector stores

Astra DB Graph

Cassandra

Cassandra Graph

Chroma DM

Clickhouse

Couchbase

Elasticsearch

FAISS

Graph RAG

Hyper-Converged Database

Local DB

Milvus

MongoDB Atlas

OpenSearch

PGVector

Pinecone

Qdrant

Redis

Supabase

Upstash

Vectara

Vectara RAG

Weaviate

6.2.2.3 Модели

6.2.2.4 Рабочие сценарии

№	Имя	Описание	Наличие	Статус	Автоматизирован	Дата изменения	Комментарий
1	Транскрибация аудио	LLM для извлечения информации из аудио записи и её последующего анализа Данный пример служит для демонстрации возможностей LLM для извлечения информации из аудио записи и её последующего анализа 1. Пользователь загружает файл в компонент Audio FIle (есть возможность сделать аналогичный компонент для загрузки audio file из URL или API эндпоинта). 2. Компонент Audio File выполняет преобразование файла в base64 строку, и передает в формате data в следующий компонент. 3. Audio Files to Text компонент выполняет подключение к MWS GPT модели и с её помощью преобразует аудио запись в текст. 4. Далее извлеченный из записи текст передается в языковую модель, куда также поступает и пользовательский вопрос, таким образом модель может отвечать на вопросы о записи.	+	done	✅	16.03.26	

№	Имя	Описание	Наличие	Статус	Автоматизирован	Дата изменения	Комментарий
		Сценарии применения: 1. Сделать аналитику записи разговора/ВКС/подкаста. 2. Автоматически извлечь из записи какие-то детали, например, договоренности, идентифицированные сложности, и т.д. --- Внимание! Для работы этого пайплайна требуется доступ к Whisper audio модели в MWS GPT - просьба согласовать с вашим менеджером доступ к этой модели.					

№	Имя	Описание	Наличие	Статус	Автоматизирован	Дата изменения	Комментарий
2	Транскрибация аудио (как есть)	Агенты, автоматически преобразующий аудио в текст и выполняющие последующую аналитику. Данный пример служит для демонстрации возможностей LLM для извлечения текста из аудио записи 1. Пользователь загружает файл в компонент Audio FIle. 2. Компонент Audio File выполняет преобразование файла в base64 строку, и передает в формате data в следующий компонент. 3. Audio Files to Text компонент выполняет подключение к MWS GPT модели и с её помощью преобразует аудио запись в текст. Сценарии применения - перевести запись в текст. --- Внимание! Для работы этого пайплайна требуется доступ к Whisper audio модели в MWS GPT - просьба согласовать с вашим менеджером доступ к этой модели.	+	done	✅	05.03.26	

№	Имя	Описание	Наличие	Статус	Автоматизирован	Дата изменения	Комментарий
3	Документация из URL	Агент, загружающий и анализирующий документы, доступные по URL. (вопрос встроен) Инструкция к запуску агента 1. Убедитесь, что во всех компонентах, которые подключаются к языковым моделям (например, "MWS GPT Model", "MWS GPT Embeddings", "Agent" и другие), указан API Base URL и API ключ для подключения к платформе (выдается нашим менеджером). Ключ выглядит как "sk-xxxxxxxxxxxxxxx". 2. Убедитесь, что во всех компонентах, которые подключаются к языковым моделям, указана Model name - модель, к которой производится обращение. Пример имени LLM модели: llama-3.3-70b-instruct. Пример имени модели для Embeddings: bge-m3 3. Если используете компонент для подключения к векторной базе данных (Vector Stores) указаны настройки для подключения. Например, для компонента FAISS указана уникальная Persist Directory для вашего сценария, например, "test1234_mycompanyname". 4. Если используете компонент File (или аналогичные для аудиофайлов и других типов файлов), то убедитесь, что в каждый такой компонент загружен нужный вам файл.	+	done	✅	05.03.26	Вопрос с импорт полями, если встать в начало и удалять, курсор идет В конец

№	Имя	Описание	Наличие	Статус	Автоматизирован	Дата изменения	Комментарий
4	Классификация обращений мед (текст, анализ)	Агент, классифицирующий медицинские обращения на основе текстового анализа. Инструкция к запуску агента 1. Убедитесь, что во всех компонентах, которые подключаются к языковым моделям (например, "MWS GPT Model", "MWS GPT Embeddings", "Agent" и другие), указан API Base URL и API ключ для подключения к платформе (выдается нашим менеджером). Ключ выглядит как "sk-xxxxxxxxxxxxxxx". 2. Убедитесь, что во всех компонентах, которые подключаются к языковым моделям, указана Model name - модель, к которой производится обращение. Пример имени LLM модели: llama-3.3-70b-instruct. Пример имени модели для Embeddings: bge-m3 3. Если используете компонент для подключения к векторной базе данных (Vector Stores) указаны настройки для подключения. Например, для компонента FAISS указана уникальная Persist Directory для вашего сценария, например, "test1234_mycompanyname". 4. Если используете компонент File (или аналогичные для аудиофайлов и других типов файлов), то убедитесь, что в каждый такой компонент загружен нужный вам файл.	+	done	✅	05.03.26	

№	Имя	Описание	Наличие	Статус	Автоматизирован	Дата изменения	Комментарий
5	Речевая аналитика-диаризация транскрибация анализ	Агент, объединяющий диаризацию, транскрибацию и анализ речи для комплексной аналитики – подходит для анализа качества работы контакт-центра. Агент Инструкция к запуску агента 1. Убедитесь, что во всех компонентах, которые подключаются к языковым моделям (например, "MWS GPT Model", "MWS GPT Embeddings", "Agent" и другие), указан API Base URL и API ключ для подключения к платформе (выдается нашим менеджером). Ключ выглядит как "sk-xxxxxxxxxxxxxxx". 2. Убедитесь, что во всех компонентах, которые подключаются к языковым моделям, указана Model name - модель, к которой производится обращение. Пример имени LLM модели: llama-3.3-70b-instruct. Пример имени модели для Embeddings: bge-m3 3. Если используете компонент для подключения к векторной базе данных (Vector Stores) указаны настройки для подключения. Например, для компонента FAISS указана уникальная Persist Directory для вашего сценария, например, "test1234_mycompanyname". 4. Если используете компонент File (или аналогичные для аудиофайлов и других типов файлов), то убедитесь, что в каждый такой компонент загружен нужный вам файл.	+	done	✅	05.03.26	

№	Имя	Описание	Наличие	Статус	Автоматизирован	Дата изменения	Комментарий
6	Контакт центр	Агент, оптимизирующий маршрутизацию и обработку обращений в контакт-центре. Пример общения пример запроса: ▪ ничего не работает ▪ мобильный интернет не работает ▪ вчера ▪ в одном месте ▪ нет проблем у знакомых ▪ +79998887766 -выключил режим в полете, все заработало, спасибо!	+	done	✅	05.03.26	
7	Деперсонализация с шифрованием	Описание Агент, который выполняет обратимое шифрование данных до отправки их во внешнюю модель, затем отправляет во внешнюю модель для обработки, получает ответ, и в ответе дешифрует данные для пользователя. Шифрование проходит прозрачно для пользователя, а обработка во внешней модели с шифрованием обеспечивает безопасность при обработке данных компании.	+	done	✅	05.03.26	Немного смущает время исполнения (уточнить)

№	Имя	Описание	Наличие	Статус	Автоматизирован	Дата изменения	Комментарий
8	Агент базовый	Функции: калькулятор, загрузка инфо из URL, получение актуальной погоды из url Инструкция к запуску агента 1. Убедитесь, что во всех компонентах, которые подключаются к языковым моделям (например, "MWS GPT Model", "MWS GPT Embeddings", "Agent" и другие), указан API Base URL и API ключ для подключения к платформе (выдается нашим менеджером). Ключ выглядит как "sk-xxxxxxxxxxxxxx". 2. Убедитесь, что во всех компонентах, которые подключаются к языковым моделям, указана Model name - модель, к которой производится обращение. Пример имени LLM модели: llama-3.3-70b-instruct. Пример имени модели для Embeddings: bge-m3 3. Если используете компонент для подключения к векторной базе данных (Vector Stores) указаны настройки для подключения. Например, для компонента FAISS указана уникальная Persist Directory для вашего сценария, например, "test1234_mycompanyname". 4. Если используете компонент File (или аналогичные для аудиофайлов и других типов файлов), то убедитесь, что в каждый такой компонент загружен нужный вам файл.	+	done	✅	13.03.26	(требуется сломать и проверить автотест)

№	Имя	Описание	Наличие	Статус	Автоматизирован	Дата изменения	Комментарий
9	Мульти-агент аэро + рейсы	Агент, демонстрирующий возможности мульти-агентной схемы для интегрирования в едином боте текущих данных о рейсах в аэропорту и информации как что-то провезти из информационной системы аэропорта. Агент Внутри 2:1 связанные агенты для помощи с полетом и авиа-расписанием	+	done	✅	05.03.26	

№	Имя	Описание	Наличие	Статус	Автоматизирован	Дата изменения	Комментарий
10	Разбор контента на изображении через VL модель	Агент, анализирующий визуальный контент изображений с помощью VLM модели.	+	done	нужен доступ до https://devapi.gpt.mws.ru/v1	05.03.26	

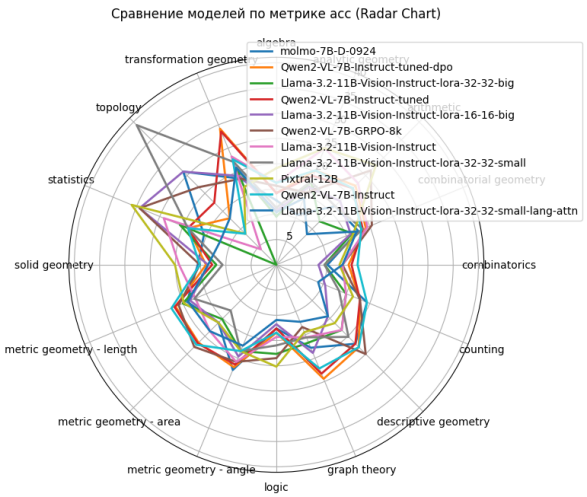
№	Имя	Описание	Наличие	Статус	Автоматизирован	Дата изменения	Комментарий
11	Деперсонализация и обработка во внешней модели	Агент, демонстрирующий деперсонализацию данных в MCP GPT и их дальнейшую обработку с помощью внешней модели.	+	done	✅	05.03.26	Попробовать более мощную модель
12	Договор парсинг + api (vl модель ocr)	Агент, автоматически извлекающий информацию из договоров с помощью VLM и API.	+	done	✅	05.03.26	Tested on llama 3.3 model.
13	RAG PGvector + file excel	Агент, использующий Postgresql+Pgvector базу данных для хранения как векторное хранилище , и позволяющий на вход подавать xlsx документы.	+	done	✅	05.03.26	
14	RAG faiss + multi-file template	Простой агент, использующий FAISS для быстрого поиска и шаблоны для обработки нескольких файлов.	+	done	✅	05.03.26	Tested on llama 3.3 model.

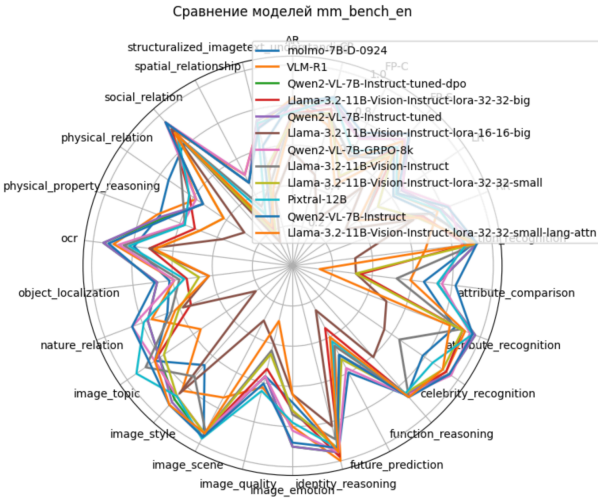
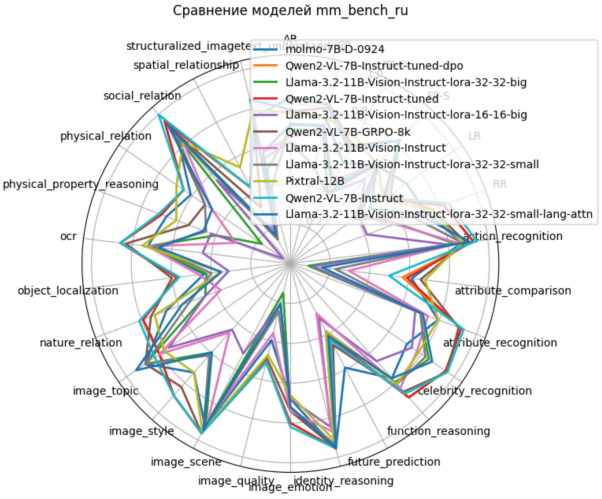
№	Имя	Описание	Наличие	Статус	Автоматизирован	Дата изменения	Комментарий
15	Multi-modal RAG from Files	Агент, извлекающий информацию из файлов, объединяя данные разных модальностей (автоматическое извлечение инфо из картинок + текста), помещение этого в базу, и обеспечение возможности ответов в чат-боте пользователям на основе инфо из текста и картинок.	+	done	✅	05.03.26	
16	Multi-modal RAG from URL	Агент, загружающий файлы через URL , объединяя данные разных модальностей (автоматическое извлечение инфо из картинок + текста), помещение этого в базу, и обеспечение возможности ответов в чат-боте пользователям на основе инфо из текста и картинок.	+	done	✅	05.03.26	Errors
17	Агент db process демо	Агент для извлечения инфо из базы данных	+	done	✅	05.03.26	
18	Агент базовый + mcp streamable http custom server	Функции: подключение к mcp серверу, калькулятор, загрузка инфо из URL, получение актуальной погоды из url	+	done	✅	05.03.26	
19	Агент базовый + mcp sse custom server	Функции: подключение к mcp серверу, калькулятор, загрузка инфо из URL, получение актуальной погоды из url	+	none	✅	10.03.26	fixed 11.03.26

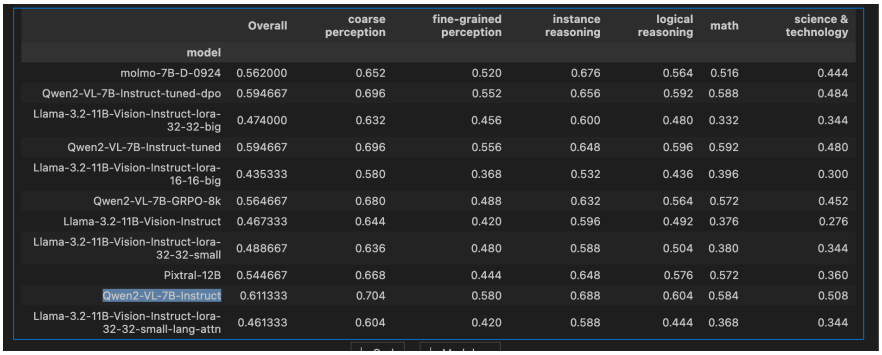
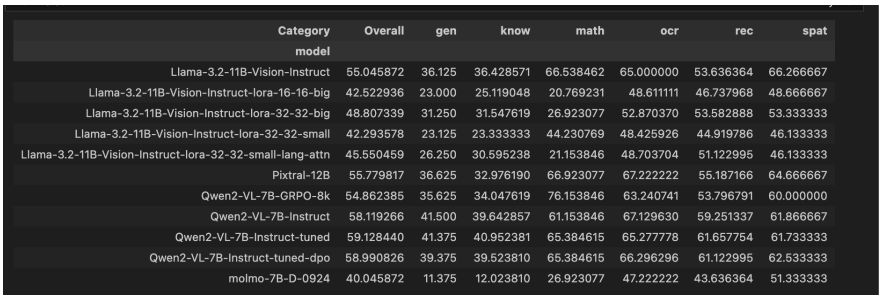
№	Имя	Описание	Наличие	Статус	Автоматизирован	Дата изменения	Комментарий
20	Договор парсинг	Агент для извлечения ключевой информации из договоров посредством обработки LLM моделью.	+	done	✅	05.03.26	

6.3 VLM

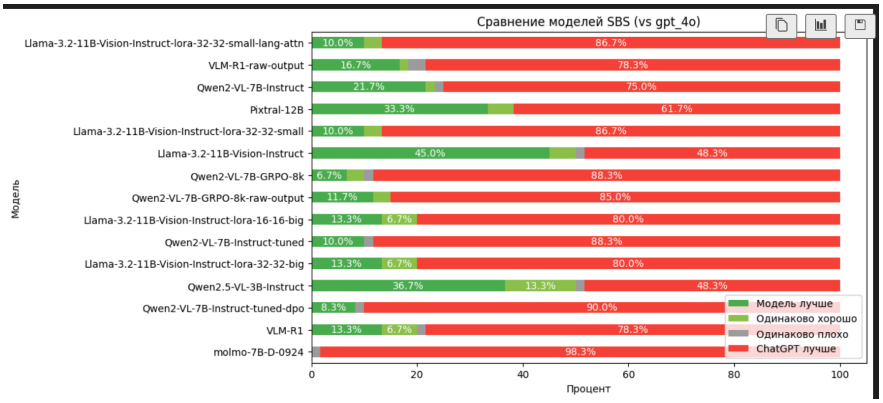
6.3.1 Бенчмаркинг

Бенчмарк	Комментарий	Язык
math_vision	Показывает как решаются различные разделы математики:  В среднем, каждая из моделей показывает достаточно хороший уровень + ответы открытые исходя из этого не самый показательный для нас бенчмарк, хотя он показывает, на сколько в целом драматично падает модель не показательный потому что закрывает одну конкретную задачу - решение математических задач	en

Бенчмарк	Комментарий	Язык
mm_bench_en	Многие модели видно, что заучиваются на бенчмарк, в целом все модели показывают высокий перформанс: 	en
mm_bench_ru	с русским картина чуть хуже, однако дообучение тоже сразу очень бросается в глаза, потому что относительно слабая для русского языка модель становится топовой: 	ru

<table>
<tr><th>Бенчмарк</th><th>Комментарий</th><th>Язык</th></tr>
<tr><td>mm_star</td><td>Хороший бенчмарк разбитый по подтемам, однако:
 
<table>
<tr><th>model</th><th>Overall</th><th>coarse perception</th><th>fine-grained perception</th><th>instance reasoning</th><th>logical reasoning</th><th>math</th><th>science & technology</th></tr>
<tr><td>molmo-7B-D-0924</td><td>0.562000</td><td>0.652</td><td>0.520</td><td>0.676</td><td>0.564</td><td>0.516</td><td>0.444</td></tr>
<tr><td>Qwen2-VL-7B-Instruct-tuned-dpo</td><td>0.594667</td><td>0.696</td><td>0.552</td><td>0.656</td><td>0.592</td><td>0.588</td><td>0.484</td></tr>
<tr><td>Llama-3.2-11B-Vision-Instruct-lora-32-32-big</td><td>0.474000</td><td>0.632</td><td>0.456</td><td>0.600</td><td>0.480</td><td>0.332</td><td>0.344</td></tr>
<tr><td>Qwen2-VL-7B-Instruct-tuned</td><td>0.594667</td><td>0.696</td><td>0.556</td><td>0.648</td><td>0.596</td><td>0.592</td><td>0.480</td></tr>
<tr><td>Llama-3.2-11B-Vision-Instruct-lora-16-16-big</td><td>0.435333</td><td>0.580</td><td>0.368</td><td>0.532</td><td>0.436</td><td>0.396</td><td>0.300</td></tr>
<tr><td>Qwen2-VL-7B-GRPO-8k</td><td>0.564667</td><td>0.680</td><td>0.488</td><td>0.632</td><td>0.564</td><td>0.572</td><td>0.452</td></tr>
<tr><td>Llama-3.2-11B-Vision-Instruct</td><td>0.467333</td><td>0.644</td><td>0.420</td><td>0.596</td><td>0.492</td><td>0.376</td><td>0.276</td></tr>
<tr><td>Llama-3.2-11B-Vision-Instruct-lora-32-32-small</td><td>0.488667</td><td>0.636</td><td>0.480</td><td>0.588</td><td>0.504</td><td>0.380</td><td>0.344</td></tr>
<tr><td>Pixtral-12B</td><td>0.544667</td><td>0.668</td><td>0.444</td><td>0.648</td><td>0.576</td><td>0.572</td><td>0.360</td></tr>
<tr><td>Qwen2-VL-7B-Instruct</td><td>0.611333</td><td>0.704</td><td>0.580</td><td>0.688</td><td>0.604</td><td>0.584</td><td>0.508</td></tr>
<tr><td>Llama-3.2-11B-Vision-Instruct-lora-32-32-small-lang-attn</td><td>0.461333</td><td>0.604</td><td>0.420</td><td>0.588</td><td>0.444</td><td>0.368</td><td>0.344</td></tr>
</table>
мы снова видим сильное дообучение некоорых моделей на бенчмарке, что не позволяет его оценивать релевантно</td><td>en</td></tr>
<tr><td>mm_vet</td><td> 
<table>
<tr><th>model</th><th>Category</th><th>Overall</th><th>gen</th><th>know</th><th>math</th><th>ocr</th><th>rec</th><th>spat</th></tr>
<tr><td>Llama-3.2-11B-Vision-Instruct</td><td>55.045872</td><td>36.125</td><td>36.428571</td><td>66.538462</td><td>65.000000</td><td>53.636364</td><td>66.266667</td></tr>
<tr><td>Llama-3.2-11B-Vision-Instruct-lora-16-16-big</td><td>42.522936</td><td>23.000</td><td>25.119048</td><td>20.769231</td><td>48.611111</td><td>46.737968</td><td>48.666667</td></tr>
<tr><td>Llama-3.2-11B-Vision-Instruct-lora-32-32-big</td><td>48.807339</td><td>31.250</td><td>31.547619</td><td>26.923077</td><td>52.870370</td><td>53.582888</td><td>53.333333</td></tr>
<tr><td>Llama-3.2-11B-Vision-Instruct-lora-32-32-small</td><td>42.293578</td><td>23.125</td><td>23.333333</td><td>44.230769</td><td>48.425926</td><td>44.919786</td><td>46.133333</td></tr>
<tr><td>Llama-3.2-11B-Vision-Instruct-lora-32-32-small-lang-attn</td><td>45.550459</td><td>26.250</td><td>30.595238</td><td>21.153846</td><td>48.703704</td><td>51.122995</td><td>46.133333</td></tr>
<tr><td>Pixtral-12B</td><td>55.779817</td><td>36.625</td><td>32.976190</td><td>66.923077</td><td>67.222222</td><td>55.187166</td><td>64.666667</td></tr>
<tr><td>Qwen2-VL-7B-GRPO-8k</td><td>54.862385</td><td>35.625</td><td>34.047619</td><td>76.153846</td><td>63.240741</td><td>53.796791</td><td>60.000000</td></tr>
<tr><td>Qwen2-VL-7B-Instruct</td><td>58.119266</td><td>41.500</td><td>39.642857</td><td>61.153846</td><td>67.129630</td><td>59.251337</td><td>61.866667</td></tr>
<tr><td>Qwen2-VL-7B-Instruct-tuned</td><td>59.128440</td><td>41.375</td><td>40.952381</td><td>65.384615</td><td>65.277778</td><td>61.657754</td><td>61.733333</td></tr>
<tr><td>Qwen2-VL-7B-Instruct-tuned-dpo</td><td>58.990826</td><td>39.375</td><td>39.523810</td><td>65.384615</td><td>66.296296</td><td>61.122995</td><td>62.533333</td></tr>
<tr><td>molmo-7B-D-0924</td><td>40.045872</td><td>11.375</td><td>12.023810</td><td>26.923077</td><td>47.222222</td><td>43.636364</td><td>51.333333</td></tr>
</table>
Аналогично и на этом бенчмарке, он разбит скорее по задачам CV направлений, нежели чем контекста вопросов, в отличие от предыдущего.

Видно заучивание моделей на ответах (более слабые модели сильнее отвечают)</td><td>en</td></tr>
</table>

Бенчмарк	Комментарий	Язык
llava bench	Слабые стороны бенчмарка: он очень маленький (всего 60 вопросов) по 3 направлениям: опиши что-то, ответь на вопрос, что за обьект на картинке? Бенчмарк на английском  |	a_better_b	both_bad	both_good	b_better_a	model vs gpt_4o	model vs gpt_4o reversed eval
---	---	---	---	---	---	---
VLM-R1	47	1	4	8	19.05 vs 80.95	17.91 vs 82.09
Qwen2.5-VL-3B-Instruct	29	1	8	22	44.78 vs 55.22	34.85 vs 65.15
Qwen2-VL-7B-Instruct-tuned	53	1	0	6	10.17 vs 89.83	11.94 vs 88.06
Qwen2-VL-7B-GRPO-8k-raw-output	51	0	2	7	14.52 vs 85.48	21.88 vs 78.12
Qwen2-VL-7B-GRPO-8k	53	1	2	4	9.84 vs 90.16	10.45 vs 89.55
Llama-3.2-11B-Vision-Instruct	29	1	3	27	48.39 vs 51.61	34.92 vs 65.08
Qwen2-VL-7B-Instruct	45	1	1	13	23.33 vs 76.67	20.9 vs 79.1
VLM-R1-raw-output	47	2	1	10	18.64 vs 81.36	23.81 vs 76.19 | Сильная сторона: это SBS бенчмарк, то есть его оценивает модель судья, которая учитывает, что изображено на картинке и оценивает ответы двух моделей	en
MMMU	Сильный бенчмарк, но снова только на английском языке	en

6.3.1.1 LLaVaRU

Основная идея - взять llava bench и перевести вопросы (не ответы) на русский язык.

Так же внедрили обратную оценку, модель оценивала только на первом месте ответ от эталонной модели, на втором - ответ от нашей модели. Тем самым получили бенчмарк:

	a_better_b	b_better_a	both_bad	both_good	model vs gpt_4o	model vs gpt_4o reversed eval
VLM-R1	50	8	2	0	13.79 vs 86.21	17.74 vs 82.26
Qwen2.5-VL-3B-Instruct	47	9	2	2	18.33 vs 81.67	16.13 vs 83.87
Qwen2-VL-7B-Instruct-tuned	54	3	3	0	5.26 vs 94.74	12.5 vs 87.5
Qwen2-VL-7B-GRPO-8k-raw-output	53	3	3	1	6.9 vs 93.1	9.84 vs 90.16
Qwen2-VL-7B-GRPO-8k	53	2	5	0	3.64 vs 96.36	11.29 vs 88.71
Llama-3.2-11B-Vision-Instruct	47	6	3	4	16.39 vs 83.61	14.75 vs 85.25
Qwen2-VL-7B-Instruct	52	7	1	0	11.86 vs 88.14	14.06 vs 85.94
VLM-R1-raw-output	50	6	3	1	12.07 vs 87.93	17.24 vs 82.76

На самом деле, это не самый корректный способ сравнивать качество модели между английским и русским бенчмарком. Поэтому нужно было разработать еще один, чтобы иметь потенциальную верхнюю границу.

6.3.1.2 LLaVaTranslated

Идея бенчмарка:

Мы берем английский входной вопрос → генерируем ответ на англ → переводим (как, чем?) → сравниваем ответы эталонной модели, полученные генерацией с русским входом.

Были замеряны два разные способа перевода:

6.3.1.2.1 GPT_3.5-turbo

	a_better_b	b_better_a	both_bad	both_good	model vs gpt_4o	model vs gpt_4o reversed eval
VLM-R1	48	9	1	2	18.03 vs 81.97	20.31 vs 79.69
Qwen2.5-VL-3B-Instruct	37	18	0	5	35.38 vs 64.62	33.33 vs 66.67
Qwen2-VL-7B-Instruct-tuned	46	12	2	0	20.69 vs 79.31	12.9 vs 87.1
Qwen2-VL-7B-GRPO-8k-raw-output	45	8	2	5	20.63 vs 79.37	14.29 vs 85.71
Qwen2-VL-7B-GRPO-8k	54	3	3	0	5.26 vs 94.74	12.9 vs 87.1
Llama-3.2-11B-Vision-Instruct	38	19	0	3	34.92 vs 65.08	35.0 vs 65.0
Qwen2-VL-7B-Instruct	45	14	1	0	23.73 vs 76.27	16.39 vs 83.61
VLM-R1-raw-output	45	12	0	3	23.81 vs 76.19	22.22 vs 77.78

6.3.1.2.2 claude-3-opus

	a_better_b	b_better_a	both_bad	both_good	model vs gpt_4o	model vs gpt_4o reversed eval
VLM-R1	46	12	1	1	21.67 vs 78.33	20.63 vs 79.37
Qwen2.5-VL-3B-Instruct	29	27	0	4	48.44 vs 51.56	41.94 vs 58.06
Qwen2-VL-7B-Instruct-tuned	45	13	2	0	22.41 vs 77.59	14.29 vs 85.71
Qwen2-VL-7B-GRPO-8k-raw-output	48	8	1	3	17.74 vs 82.26	11.48 vs 88.52
Qwen2-VL-7B-GRPO-8k	54	4	2	0	6.9 vs 93.1	9.68 vs 90.32
Qwen2-VL-7B-Instruct	43	16	1	0	27.12 vs 72.88	19.35 vs 80.65
VLM-R1-raw-output	43	14	0	3	26.98 vs 73.02	30.0 vs 70.0

Еще была итерация перевода с deepseek, но дип сик зачастую в перевод вставлял иероглифы.

Выводы: это наша потенциальная граница. Из интересного:

1. Модель

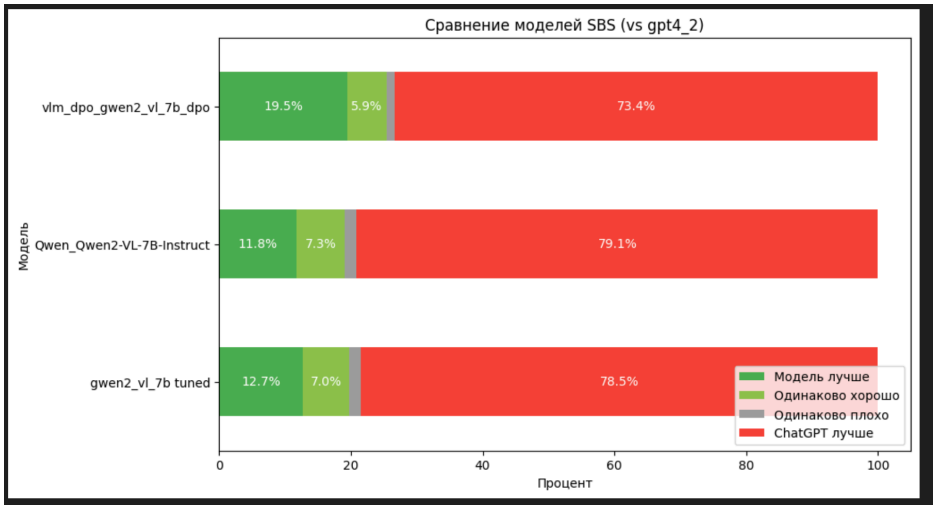
   Qwen2-VL-7B-Instruct-tuned

   в русском и английском бенчмарке драматично падает по результату ответа по сравнению с байзлайн моделью, однако в бенчмарке переводном, она показывает очень сходное качество, вне зависимости от перевода - это может говорить о том, что перевод улучшает качество, выбрасывает лишнее и как-то редактирует текст

2. перевод

   claude-3-opus оценивается gpt-4o (эталонная модель) в среднем выше, чем перевод gpt-3.5-turbo. Это может быть связано с тем, что первая модель выше в рейтинге lmsys и умнее

6.3.1.3 Text-only benchmark:



Картина которую мы наблюдаем, показывает, что при адаптации, содержащей текстовые данные мы видим улучшение метрик базовой модели на нашем бенчмарке. Однако из-за этого драматично падает качество на vision задачах. Поэтому на данный момент отдается сильное внимание результатам именно визуальных бенчмарков в совокупности с SBS оценкой.

6.3.2 Исследования

- [ ] Датасет, где одинаковые картинки имеют вопросы разные на 2ух языках
- [ ] большая лора (256 на 256) на все внимание и MLP (визуал-мерж-текст)

- [ ] Одинаковые и неодинаковые вопросы на одну картинку

6.3.3 Таблица экспериментов

id	модель	бенчмарки	комментарий
1	Llama-3.2-11B-Vision-Instruct	LLavaBench 48.39/34.92 LlavaBenchRU 16.39/14.75 TranslatedLLavaBench gpt3.5: 34.92 / 35.0 claude:	бейзлайн
2	Qwen2.5-VL-3B-Instruct	LLavaBench 44.78/34.85 LlavaBenchRU 18.33/16.13 TranslatedLLavaBench gpt3.5: 35.38 / 33.33 claude:	бейзлайн
3	VLM-R1	LLavaBench 19.05/17.91 LlavaBenchRU 13.79/17.74 TranslatedLLavaBench gpt3.5: 18.03 / 20.31 claude:	бейзлайн - R1 поверх **пункта 2**

id	модель	бенчмарки	комментарий
4	VLM-R1-raw-output	LLavaBench 18.64/23.81 LlavaBenchRU 12.07/17.24 TranslatedLLavaBench gpt3.5: 23.81 / 22.22 claude: 26.98 / 30.0	бейзлайн - R1 поверх **пункта 2** + оценка вместе с <think>
5	Qwen2-VL-7B-Instruct	LLavaBench 23.33/20.9 LlavaBenchRU 11.86/14.06 TranslatedLLavaBench gpt3.5: 23.73/16.39 claude:	бейзлайн
6	Qwen2-VL-7B-GRPO-8k-raw-output	LLavaBench 14.52/21.88 LlavaBenchRU 6.9 /9.84 TranslatedLLavaBench gpt3.5: 20.63/14.29 claude: 17.74/ 11.48	бейзлайн - R1 поверх **пункта 5** + оценка вместе с <think>

id	модель	бенчмарки	комментарий
7	Qwen2-VL-7B-GRPO-8k	LLavaBench 9.84/10.45 LlavaBenchRU 3.64/11.29 TranslatedLLavaBench gpt3.5: 5.26 /12.9 claude: 6.9 / 3.45	бейзлайн - R1 поверх **пункта 5**
8	idefics fine-tune	LLavaBench 0.0/9.23 LlavaBenchRU 5.26/6.56 TranslatedLLavaBench gpt3.5: 7.02/8.2 claude: 6.78/6.67	
9	Qwen2.5-VL-7B-Instruct	LLavaBench 54.41/47.06 LlavaBenchRU 43.08/30.65 TranslatedLLavaBench gpt3.5: 74.19/ 55.38 claude: 76.56/ 53.12	
10	VL-Rethinker-7B	LLavaBench 63.93/53.33 LlavaBenchRU TranslatedLLavaBench gpt3.5: 81.67/ 66.67 claude: 81.67/60.0	GRPO_SSR от Qwen2.5-VL-7B-Instruct

id	статус	модель	данные SFT	данные DPO	лора(префикс) параметры	бенчмарки	комментарий
1	завершено	Qwen2-VL-7B-Instruct	"the_cauldron", "LLaVA_Instruct_ru", "GQA", "ocra_ru_mtsai_50k", "gusev", "spellcheck_benchmark", "ocra_ru_summary_mtsai_50k", "gsm8k-ru", "spellcheck_benchmark", "wikiomnia", "CodeExercise_Python_27k_ru", "HC3_ru", "logic_tasks_ru", "samsum_ru"		"lora": { "r": 32, "lora_alpha": 32, "target_modules": ["q_proj", "v_proj", "k_proj", "o_proj", "mlp.0", "mlp.2"], "bias": "none", "lora_dropout": 0.05, "task_type": "CAUSAL_LM", "use_dora": false, "init_lora_weights": "gaussian" }, num_virtual_tokens=20	LlamaBench 10.17 / 11.94 LlamaBenchRu 5.26 / 12.5 TranslatedLLavaBench gpt3.5: 20.69 / 12.9 claude: 22.41 / 14.29	повтор пайплайна для MTS ANYA

id	статус	модель	данные SFT	данные DPO	лора(префикс) параметры	бенчмарки	комментарий
2	завершено	Qwen2-VL-7B-Instruct	**"the_cauldron"**, **"LLaVA_Instruct_ru"**, **"GQA"**, "ocra_ru_mtsai_50k", "gusev", "spellcheck_benchmark", "ocra_ru_summary_mtsai_50k", "gsm8k-ru", "spellcheck_benchmark", "wikiomnia", "CodeExercise_Python_27k_ru", "HC3_ru", "logic_tasks_ru", "samsum_ru"	"0x7o/fialka-v2-dpo", "0x7o/oasst-ru-dpo-v1-rm", "argilla/distilabel-math-preference-dpo", "argilla/dpo-mix-7k", **"llamafactory/RLHF-V"**	"train_arg": { "warmup_steps": 3, "per_device_train_batch_size": 2, "per_device_eval_batch_size": 2, "gradient_accumulation_steps": 64, "eval_accumulation_steps": 64, "num_train_epochs": 1, "fp16": true, "bf16": false, "learning_rate": 2.5e-05, "optim": "adamw_torch", "logging_steps": 5, "save_strategy": "steps", "save_steps": 500, "eval_strategy": "steps", "eval_steps": 500, "do_eval": true }, "max_length": 8124	LlamaBench 8.47 / LlamaBenchRU 6.9/	DPO над пунктом 1 повтор пайплайна для MTS ANYA

id	статус	модель	данные SFT	данные DPO	лора(префикс) параметры	бенчмарки	комментарий
					"lora": { "target_modules": ["fc1", "qkv", "proj", "fc2", "q_proj", "v_proj", "k_proj", "o_proj", "mlp.0", "mlp.2", "lm_head", "down_proj", "up_proj", "gate_proj"] },		

3	завершено	Llama-3.2-11B-Vision-Instruct	**"the_cauldron"**,
			"GQA",
			"ocra_ru_mtsai_50k",
			"gusev",
			"spellcheck_benchmark",
			"ocra_ru_summary_mtsai_50k",
			"gsm8k-ru",
			"samsum_ru",
			"LLaVA_Instruct_ru"
					"lora": {
					"r": 16,
					"lora_alpha": 16,
					"target_modules": [
					"q_proj",
					"v_proj",
					"k_proj",
					"o_proj"
					],
					"bias": "none",
					"lora_dropout": 0,
					"task_type": "CAUSAL_LM"
					}
						LLamaBench 21.31/21.13
						
						LlamaBenchRU 8.77/9.84
						gpt: 20.0/16.92
						claude: 15.25 / 18.46
							Итерация с ограниченным числом данных при малых параметрах и чисто лора, потому что Prefix падает с ошибкой

id	статус	модель	данные SFT	данные DPO	лора(префикс) параметры	бенчмарки	комментарий
4	завершено	Llama-3.2-11B-Vision-Instruct	**"the_cauldron", "GQA",** "ocra_ru_mtsai_50k", "gusev", "spellcheck_benchmark", "ocra_ru_summary_mtsai_50k", "gsm8k-ru", "samsum_ru", **"LLaVA_Instruct_ru"**		"lora": { "r": 32, "lora_alpha": 32, "target_modules": ["q_proj", "v_proj", "k_proj", "o_proj"], "bias": "none", "lora_dropout": 0.05, "task_type": "CAUSAL_LM" }	LLamaBench 15.87/25.0 LlamaBenchRU 10.17/12.7 gpt: 18.64 / 15.62 claude: 16.95 / 14.52	
5	завершено	Llama-3.2-11B-Vision-Instruct	["the_cauldron", "GQA", "LLaVA_Instruct_ru"]		"lora": { "r": 32, "lora_alpha": 32, "target_modules": ["q_proj", "v_proj", "k_proj", "o_proj"], "bias": "none", "lora_dropout": 0.05, "task_type": "CAUSAL_LM" }	LLamaBench 24.19/21.13 LlamaBenchRU 10.17/12.7 gpt: 18.33/14.52 claude: 20.0/11.29	пункт 4: поменьше датасет

id	статус	модель	данные SFT	данные DPO	лора(префикс) параметры	бенчмарки	комментарий

6	завершено	Llama-3.2-11B-Vision-Instruct	[
			"the_cauldron",
			"GQA",
			"LLaVA_Instruct_ru"
			]
					"lora": {
					"r": 32,
					"lora_alpha": 32,
					"target_modules": [
					"q_proj",
					"v_proj",
					"k_proj",
					"o_proj"
					],
					"bias": "none",
					"lora_dropout": 0.05,
					"task_type": "CAUSAL_LM"
					}
					только внимание
						LLamaBench 14.29/20.0
						
						LlamaBenchRU 10.34/11.29
						
						gpt: 10.34/12.7
						claude: 10.17/14.06
							пункт 5, но только внимание

7	завершено	Qwen2.5-VL-3B-Instruct	"LLaVA_Instruct_ru"		"lora": { "r": 16, "lora_alpha": 16, "target_modules": ["q_proj", "v_proj", "k_proj", "o_proj"], "bias": "none", "lora_dropout": 0.05, "task_type": "CAUSAL_LM" }, p_tuning num_virtual_tokens=30	положительных результатов нет, модель предсказывала один токен	

id	статус	модель	данные SFT	данные DPO	лора(префикс) параметры	бенчмарки	комментарий
8	завершено	Qwen2.5-VL-3B-Instruct	"LLaVA_Instruct_ru"		"lora": { "r": 16, "lora_alpha": 16, "target_modules": ["q_proj", "v_proj", "k_proj", "o_proj"], "bias": "none", "lora_dropout": 0.05, "task_type": "CAUSAL_LM" }, prompt_tuning num_virtual_tokens=30	положительных результатов нет, модель предсказывала один токен	
9	завершено	Qwen2-VL-7B-Instruct	"LLaVA_Instruct_ru"		"lora": { "r": 16, "lora_alpha": 16, "target_modules": ["q_proj", "v_proj", "k_proj", "o_proj"], "bias": "none", "lora_dropout": 0.05, "task_type": "CAUSAL_LM" }, prefix_tuning num_virtual_tokens=30	положительных результатов нет, модель предсказывала мракобесие	отдельный пайплайн, сначала префикс, потом лора, все по статье

id	статус	модель	данные SFT	данные DPO	лора(префикс) параметры	бенчмарки	комментарий
10	завершено	Qwen2-VL-7B-Instruct	"ocra_ru_mtsai_50k", "gusev", "spellcheck_benchmark", "ocra_ru_summary_mtsai_50k", "gsm8k-ru", "samsum_ru",		"lora": { "r": 32, "lora_alpha": 32, "target_modules": ["q_proj", "v_proj", "k_proj", "o_proj", "gate_proj", "up_proj", "down_proj"], "bias": "none", "lora_dropout": 0.05, "task_type": "CAUSAL_LM", "use_dora": false, "init_lora_weights": "gaussian" }, num_virtual_tokens=20 учим текст декодер внимание + mlp	LLamaBench 52.94/39.71 LlamaBenchRU 24.59/21.88 gpt: 55.38/37.31 claude:60.94/40.3	полный пайпани, без визуальных данных

id	статус	модель	данные SFT	данные DPO	лора(префикс) параметры	бенчмарки	комментарий
11	завершено	Qwen2-VL-7B-Instruct	"ocra_ru_mtsai_50k", "gusev", "spellcheck_benchmark", "ocra_ru_summary_mtsai_50k", "gsm8k-ru", "samsum_ru",		"lora": { "r": 32, "lora_alpha": 32, "target_modules": ["q_proj", "v_proj", "k_proj", "o_proj",], "bias": "none", "lora_dropout": 0.05, "task_type": "CAUSAL_LM", "use_dora": false, "init_lora_weights": "gaussian" }, num_virtual_tokens=20 учим только текст декодер внимание	LLamaBench 47.14/39.13 LlamaBenchRU 24.59/22.22 gpt: 56.25 / 40.3 claude: 69.23/40.9	

id	статус	модель	данные SFT	данные DPO	лора(префикс) параметры	бенчмарки	комментарий
12	завершено	Qwen2-VL-7B-Instruct	"ocra_ru_mtsai_50k", "gusev", "spellcheck_benchmark", "ocra_ru_summary_mtsai_50k", "gsm8k-ru", "samsum_ru" **"GQA", "optic_QA_pdf_russian_distill", "text_meme_distill"**		"lora": { "r": 32, "lora_alpha": 32, "target_modules": ["q_proj", "v_proj", "k_proj", "o_proj", "gate_proj", "up_proj", "down_proj" "mlp.0", "mlp.2"], "bias": "none", "lora_dropout": 0.05, "task_type": "CAUSAL_LM", "use_dora": false, "init_lora_weights": "gaussian" }, num_virtual_tokens=20 учим все: проджектор текст декодер	LLamaBench 44.93/40.0 LlamaBenchRU 24.59/22.22 gpt: 59.38/36.36 claude: 62.5/39.39	полный пайпани с картинками, сравнение с пунктом 1

id	статус	модель	данные SFT	данные DPO	лора(префикс) параметры	бенчмарки	комментарий
13	завершено	Qwen2.5-VL-3B-Instruct	"LLaVA_Instruct_ru"		"lora": { "r": 16, "lora_alpha": 16, "target_modules": ["q_proj", "v_proj", "k_proj", "o_proj"], "bias": "none", "lora_dropout": 0.05, "task_type": "CAUSAL_LM" }, lora merger + text	LLamaBench 38.1/29.85 LlamaBenchRU 8.33/11.11 gpt: 41.27 / 31.75 claude: 43.75/31.75	

id	статус	модель	данные SFT	данные DPO	лора(префикс) параметры	бенчмарки	комментарий
14	завершено	Qwen2.5-VL-3B-Instruct	"LLaVA_Instruct_ru"		"lora": { "r": 16, "lora_alpha": 16, "target_modules": ["q_proj", "v_proj", "k_proj", "o_proj"], "bias": "none", "lora_dropout": 0.05, "task_type": "CAUSAL_LM" }, lora visual + merger + text	LLamaBench 40.62/28.36 LlamaBenchRU 6.9/11.48 gpt: 40.62/28.57 claude: 45.16/32.26	

id	статус	модель	данные SFT	данные DPO	лора(префикс) параметры	бенчмарки	комментарий

16	нужны подготовительные работы по данны	Qwen2.5-VL-3B-Instruct	"LLaVA_Instruct_ru"
			
			+ какие-то хорошие данные
					"lora": {
					"r": 256,
					"lora_alpha": 256,
					"target_modules": [
					"q_proj",
					"v_proj",
					"k_proj",
					"o_proj"
					],
					"bias": "none",
					"lora_dropout": 0.05,
					"task_type": "CAUSAL_LM"
					},
					lora
					`visual + merger + text`		

17	завершено	Qwen2.5-VL-3B-Instruct	"xgqa" - параллельные данные		"lora": { "r": 16, "lora_alpha": 16, "target_modules": ["q_proj", "v_proj", "k_proj", "o_proj"], "bias": "none", "lora_dropout": 0.05, "task_type": "CAUSAL_LM" }, lora merger + text	LLamaBench 54.69 / 45.16 LlamaBenchRU 30.0 / 23.33 gpt: 60.94/49.18 claude: 64.62/55.74	

id	статус	модель	данные SFT	данные DPO	лора(префикс) параметры	бенчмарки	комментарий
18	завершено	Qwen2.5-VL-3B-Instruct	"xgqa" - разные данные		"lora": { "r": 16, "lora_alpha": 16, "target_modules": ["q_proj", "v_proj", "k_proj", "o_proj"], "bias": "none", "lora_dropout": 0.05, "task_type": "CAUSAL_LM" }, lora merger + text	LLamaBench h 50.0/43.75 LlamaBenchRU 16.95/16.39 gpt: 57.81/44.26 claude: 61.9 /47.54	
19	завершено	Qwen2.5-VL-3B-Instruct	"GQA", "optic_QA_pdf_russian_distill", "text_meme_distill"		"lora": { "r": 16, "lora_alpha": 16, "target_modules": ["q_proj", "v_proj", "k_proj", "o_proj"], "bias": "none", "lora_dropout": 0.05, "task_type": "CAUSAL_LM" }, lora merger + text	LLamaBench h 57.35/53.73 LlamaBenchRU 41.38/26.23 gpt: 67.19/52.38 claude: 69.7/57.38	

id	статус	модель	данные SFT	данные DPO	лора(префикс) параметры	бенчмарки	комментарий
20	завершено	Qwen2-VL-7B-Instruct	"ocra_ru_mtsai_50k", "gusev", "spellcheck_benchmark", "ocra_ru_summary_mtsai_50k", "gsm8k-ru", "samsum_ru" **"GQA", "optic_QA_pdf_russian_distill", "text_meme_distill"**		"lora": { "r": 32, "lora_alpha": 32, "target_modules": ["q_proj", "v_proj", "k_proj", "o_proj", "gate_proj", "up_proj", "down_proj" "mlp.0", "mlp.2"], "bias": "none", "lora_dropout": 0.05, "task_type": "CAUSAL_LM", "use_dora": false, "init_lora_weights": "gaussian" }, num_virtual_tokens=20 учим все: проджектор текст декодер	LLamaBench 47.83/36.36 LlamaBenchRU 20.97/17.19 gpt: 47.62 / 29.85 claude: 53.97/40.3	с системным промптом, обученная итерация от 12

id	статус	модель	данные SFT	данные DPO	лора(префикс) параметры	бенчмарки	комментарий
21	завершено	Qwen2-VL-7B-Instruct	"ocra_ru_mtsai_50k", "gusev", "spellcheck_benchmark", "ocra_ru_summary_mtsai_50k", "gsm8k-ru", "samsum_ru" **"GQA",** **"optic_QA_pdf_russian_distill", "text_meme_distill"** **"xgqa_distill"**		"lora": { "r": 32, "lora_alpha": 32, "target_modules": ["q_proj", "v_proj", "k_proj", "o_proj", "gate_proj", "up_proj", "down_proj" "mlp.0", "mlp.2"], "bias": "none", "lora_dropout": 0.05, "task_type": "CAUSAL_LM", "use_dora": false, "init_lora_weights": "gaussian" }, num_virtual_tokens=20 учим все: визуал проджектор текст декодер	LLamaBench 44.29/39.71 LlamaBenchRU 27.42/18.75 gpt: 60.0/35.82 claude: 69.23/40.91	

id	статус	модель	данные SFT	данные DPO	лора(префикс) параметры	бенчмарки	комментарий
22	завершено	Qwen2.5-VL-3B-Instruct	"GQA", "optic_QA_pdf_russian_distill", "text_meme_distill" **"xgqa_distill"** **не полное**		"lora": { "r": 16, "lora_alpha": 16, "target_modules": ["q_proj", "v_proj", "k_proj", "o_proj"], "bias": "none", "lora_dropout": 0.05, "task_type": "CAUSAL_LM" }, lora merger + text	LLamaBench 56.06/46.15 LlamaBenchRU 33.9/33.9 gpt: 70.31/57.14 claude: 67.19/58.06	
23	оценивается	Qwen2.5-VL-3B-Instruct	"GQA", "optic_QA_pdf_russian_distill", "text_meme_distill" **"xgqa_distill"** **полное**		"lora": { "r": 16, "lora_alpha": 16, "target_modules": ["q_proj", "v_proj", "k_proj", "o_proj"], "bias": "none", "lora_dropout": 0.05, "task_type": "CAUSAL_LM" }, lora merger + text	LLamaBench 58.82/50.0 LlamaBenchRU 32.79/32.26 gpt: 66.13/55.56 claude: 70.31/62.3	

id	статус	модель	данные SFT	данные DPO	лора(префикс) параметры	бенчмарки	комментарий
24	оценивается	Qwen2.5-VL-7B-Instruct	"GQA", "optic_QA_pdf_russian_distill", "text_meme_distill" **"xgqa_distill"** **полное**		"lora": { "r": 16, "lora_alpha": 16, "target_modules": ["q_proj", "v_proj", "k_proj", "o_proj"], "bias": "none", "lora_dropout": 0.05, "task_type": "CAUSAL_LM" }, lora merger + text	LLamaBench 63.64/62.12 LlamaBenchRU 56.67/50.79 gpt: 82.26/79.03 claude: 85.25/75.41	
25	завершено	Qwen2.5-VL-3B-Instruct	"GQA", "optic_QA_pdf_russian_distill", "text_meme_distill" **"xgqa_distill"** **не полное**		"lora": { "r": 16, "lora_alpha": 16, "target_modules": ["q_proj", "v_proj", "k_proj", "o_proj"], "bias": "none", "lora_dropout": 0.05, "task_type": "CAUSAL_LM" }, lora merger + text	LLamaBench 60.29/53.85 LlamaBenchRU 26.32/23.73 gpt: 66.67/61.29 claude: 75.81/57.38	выбрасывание больших градиентов 5%

id	статус	модель	данные SFT	данные DPO	лора(префикс) параметры	бенчмарки	комментарий
26	завершено	Qwen2.5-VL-3B-Instruct	"GQA", "optic_QA_pdf_russian_distill", "text_meme_distill"		"lora": { "r": 16, "lora_alpha": 16, "target_modules": ["q_proj", "v_proj", "k_proj", "o_proj"], "bias": "none", "lora_dropout": 0.05, "task_type": "CAUSAL_LM" }, lora merger + text	LLamaBench 55.88/49.25 LlamaBenchhRU 29.31/15.0 gpt: 59.38/50.0 claude: 61.54/53.23	выбрасывание больших градиентов 5%
27	завершено	Qwen2.5-VL-3B-Instruct		"0x7o/fialka-v2-dpo", "0x7o/oasst-ru-dpo-v1-rm", "argilla/distilabel-math-preference-dpo", "argilla/dpo-mix-7k", **"llamafactory/RLHF-V"**		LLamaBench 38.81/37.31 LlamaBenchhRU 16.67/16.13 gpt: 47.69/36.51 claude: 49.23/0	DPO

id	статус	модель	данные SFT	данные DPO	лора(префикс) параметры	бенчмарки	комментарий
28	завершено	Qwen2.5-VL-3B-Instruct		"llamafactory/RLHF-V"		LLamaBench 45.45/40.91 LlamaBenchRU 18.64/13.33 gpt: 47.69/37.1 claude: 50.0/40.32	DPO

6.4 Анализ задержки запуска Flow в Langflow

> **Контекст**
>
> Между нажатием кнопки для запуска пайплайна в UI и началом фактического выполнения компонентов на бэкенде наблюдается задержка 10+ секунд. Ниже представлен детальный анализ причин и предложения по оптимизации.

6.4.1 Общая схема выполнения

```
[FRONTEND]                                          [BACKEND]
Нажатие кнопки Play/Send
    |
    ▼
1. sendMessage()
2. buildFlow() - валидация узлов/рёбер
3. POST /build/{flowId}/flow  ──────────►  4. start_flow_build()
                                            5. Создание очереди + запуск
задачи
    ◄── возврат job_id ──────────────────
6. GET /build/{jobId}/events  ──────────►  7. Открытие streaming-соединения
                                            8. build_graph_and_get_order():
                                               8a. Запрос flow из БД
                                               8b. Graph.from_payload()
```

```
                                                    8c. graph.initialize_run()
                                                    8d. sort_vertices()
                                                    8e. Кэширование графа
                                                9. Событие on_vertices_sorted
                                                10. build_vertices() -
выполнение
```

6.4.2 Выявленные проблемы

6.4.2.1 1. Граф пересоздаётся с нуля при каждом запуске

Оценка влияния: примерно 2–5 секунд на запуск

Где смотреть:

- функция `build_graph_and_get_order()`
- функция `build_graph_from_db()`

Описание:
При каждом нажатии кнопки запуска вызывается цепочка:
`build_graph_and_get_order()` → `create_graph()` → `build_graph_from_db()` → `build_graph_from_db_no_cache()` → `build_graph_from_data()` → `Graph.from_payload()`

Кэш (`chat_service.set_cache`) записывается **после** построения графа, но **никогда не проверяется перед построением**. То есть даже для неизменённого flow граф строится заново каждый раз.

Это включает:

- Deep copy JSON-данных flow
- Создание объектов Vertex для всех узлов
- Создание и валидация объектов Edge
- Инстанциирование компонентов
- Топологическая сортировка

Предложения по решению:

1. **Реализовать проверку кэша перед построением графа.** В `build_graph_and_get_order()` перед вызовом `create_graph()` проверять, существует ли в `chat_service` валидный кэшированный граф для данного `flow_id` . Использовать хеш `flow.data` (или `flow.updated_at`) для инвалидации.

2. **Инкрементальное обновление графа.** Метод `Graph.update()` (`base.py:1199`) уже существует и позволяет сравнивать вершины. Можно использовать его для обновления только изменённых узлов вместо пересоздания всего графа.
3. **Добавить версионирование flow.** При сохранении flow записывать хеш данных. При запуске сравнивать хеш - если не изменился, использовать кэшированный граф.

6.4.2.2 2. Deep copy всех данных flow

Оценка влияния: 100–500 мс

Где смотреть:

- utils.py:87

Описание:

```
def process_flow(flow_object):
    cloned_flow = copy.deepcopy(flow_object)  # <-- полная глубокая копия
    ...
```

Функция `process_flow()` выполняет `copy.deepcopy` всей JSON-структуры flow, включающей все узлы, рёбра, шаблоны и параметры. Для сложных флоу с 20+ компонентами и объёмными шаблонами это может занимать сотни миллисекунд.

Предложения по решению:

1. **Заменить deep copy на shallow copy с точечным копированием.** Выполнять deep copy только для тех узлов, которые реально модифицируются в `process_node()` (узлы с вложенными flow/группами). Остальные узлы копировать поверхностно.
2. **Оптимизировать через** `msgspec` **или** `orjson`**.** Сериализация + десериализация через быструю библиотеку может быть быстрее, чем `copy.deepcopy` для больших структур.
3. **Выполнять** `process_flow` **один раз при сохранении flow**, а не при каждом запуске. Сохранять «развёрнутый» (ungrouped) вариант flow в БД.

6.4.2.3 3. Последовательное инстанциирование компонентов

Оценка влияния: примерно500–3000 мс

Где смотреть:

- функция `_instantiate_components_in_vertice`

Описание:

```
def _instantiate_components_in_vertices(self) -> None:
    for vertex in self.vertices:
        vertex.instantiate_component(self.user_id)
```

Для каждой вершины последовательно вызывается `instantiate_component()`, который:

- Извлекает код компонента из параметров
- Вызывает `eval_custom_component_code(code)` - компиляция Python-кода
- Создаёт экземпляр класса компонента с параметрами

Для флоу с N компонентами это N последовательных вызовов eval + создание объекта.

Предложения по решению:

1. **Кэширование скомпилированных классов компонентов.** Результат `eval_custom_component_code(code)` - это класс Python. Можно кэшировать его по хешу исходного кода, чтобы не перекомпилировать при каждом запуске.
2. **Параллельное инстанциирование.** Если между компонентами нет зависимостей на этапе инстанциирования (а их нет - зависимости разрешаются позже при build), можно использовать `concurrent.futures.ThreadPoolExecutor` для параллельного создания (будет сложность при интеграции решения с Celery).
3. **Предварительная компиляция при загрузке компонентов.** При регистрации компонента в системе компилировать код один раз и сохранять объект класса в реестре.

6.4.2.4 4. Двухфазный HTTP-запрос на фронтенде

Оценка влияния: 100–300 мс

Где смотреть:

- - `src/frontend/src/utils/buildUtils.ts`
- - `src/frontend/src/stores/flowStore.ts`

Описание:
Фронтенд выполняет два последовательных HTTP-запроса:

1. {{POST /build/
   Unknown macro: {flowId}
   /flow}} → получение `job_id`

2. {{GET /build/
   Unknown macro: {jobId}
   /events}} → открытие streaming-соединения

Второй запрос может начаться только после завершения первого, что добавляет полный HTTP round-trip.

Предложения по решению:

1. **Использовать режим DIRECT.** В этом режиме (`EventDeliveryType.DIRECT`) POST-запрос сразу возвращает streaming-ответ, без промежуточного `job_id` . Уже реализовано в коде, но не используется по умолчанию.
2. **WebSocket-соединение.** Заменить HTTP-запросы на постоянное WebSocket-соединение для обмена событиями. Это исключит overhead на установку соединения.
3. **Параллельная инициализация.** Начинать подготовку streaming-соединения одновременно с отправкой POST-запроса, подключаясь к очереди событий сразу после получения `job_id` .

6.4.2.5 5. Загрузка flow из БД (большой JSON-блоб)

Оценка влияния: 50–200 мс

Где смотреть:

- функция `build_graph_from_db_no_cache`

Описание:

```
async def build_graph_from_db_no_cache(flow_id, session, **kwargs):
    flow = await session.get(Flow, flow_id)  # <-- весь flow.data
загружается
```

Вся структура flow (узлы, рёбра, шаблоны, параметры) хранится как единый JSON-блоб в поле `flow.data` . Для сложных flow это может быть десятки-сотни килобайт данных.

Предложения по решению:

1. **Кэширование flow.data в памяти.** Использовать in-memory кэш (например, `cachetools.TTLCache`) для хранения данных flow, инвалидируя по `flow.updated_at` .
2. **Сжатие данных flow.** Хранить `flow.data` в сжатом виде (gzip/zstd) в БД, распаковывать при чтении. Это уменьшит I/O и время передачи.

6.4.2.6 6. Инициализация сервиса трассировки

Оценка влияния: 0–500 мс (зависит от конфигурации)

Где смотреть:

- функция `initialize_run`

Описание:

```
async def initialize_run(self) -> None:
    if self.tracing_service:
        await self.tracing_service.start_tracers(
            run_id=uuid.UUID(self._run_id),
            run_name=run_name,
            user_id=self.user_id,
            session_id=self.session_id,
        )
```

При каждом запуске инициализируются трассировщики (LangSmith, LangWatch и т.д.), что может включать сетевые вызовы к внешним сервисам.

Предложения по решению:

1. **Ленивая инициализация трассировщиков.** Устанавливать соединение с внешним сервисом трассировки один раз при старте приложения, а не при каждом запуске flow.
2. **Асинхронная инициализация в фоне.** Запускать `start_tracers()` как фоновую задачу, не блокируя основной пайплайн выполнения.

6.5 Как работают связи компонентов в воркфлоу Langflow и почему они ломаются при обновлении версии

6.5.1 1. Как устроены связи компонентов

6.5.1.1 1.1 Модель данных

Каждая связь между двумя компонентами - это **edge**, который хранится как JSON. Edge содержит сериализованные метаданные "handle" для source (выход) и target (вход).

Source handle (правая сторона, выход компонента):